

Konvolučné neurónové siete,

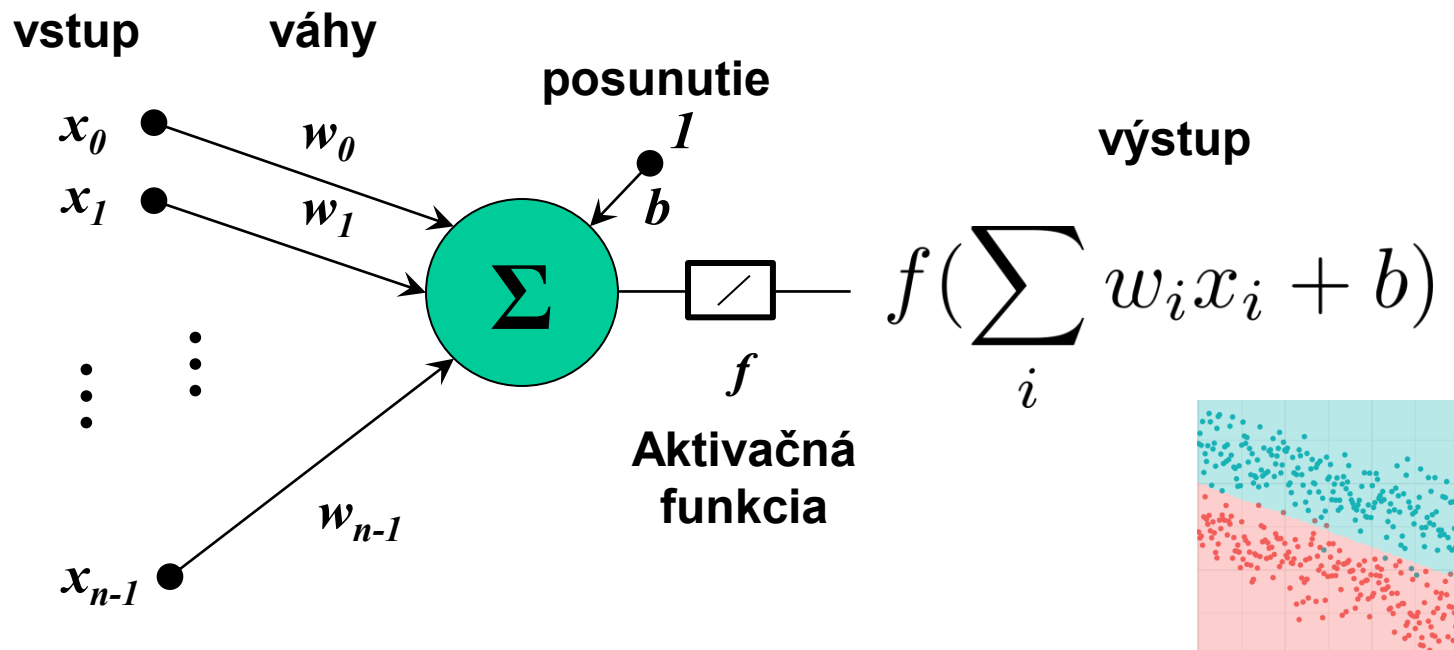
29.10.2025, 15:00-17:00

Andrej Lúčny

Reprezentácia obrazu. Spracovanie obrazu pomocou kernelov. Implementácia kernelov pomocou konvolučnej neurónovej siete. Implementácia paralelne bežiacich perceptronov zdieľajúcich váhy pomocou konvolučných vrstiev.

<https://github.com/andy1ucny/OAI>

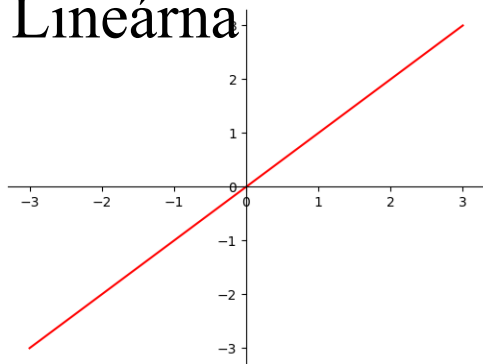
Neurón – Lineárny regressor



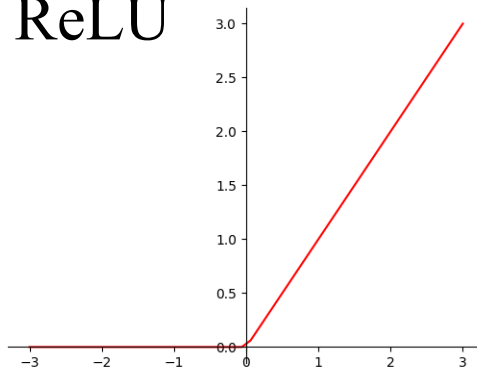
- Neurón počíta skalárny súčin vstupu s váhami, pripočíta posunutie a aplikuje aktivačnú funkciu
- Aktivačná funkcia môže byť: lineárna, sigmoida, hyperbolický tangens, ...

Úvod: Aktivačné funkcie

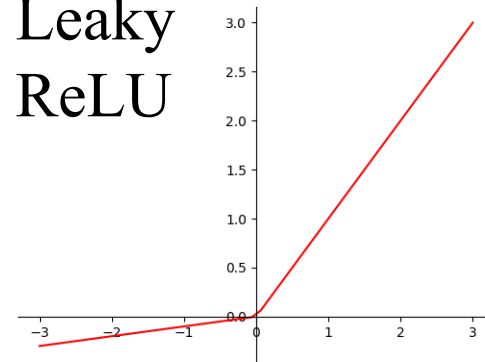
Lineárna



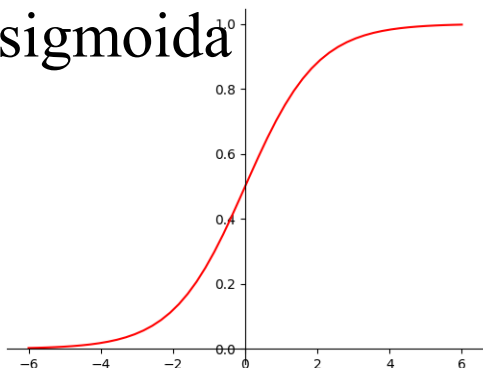
ReLU



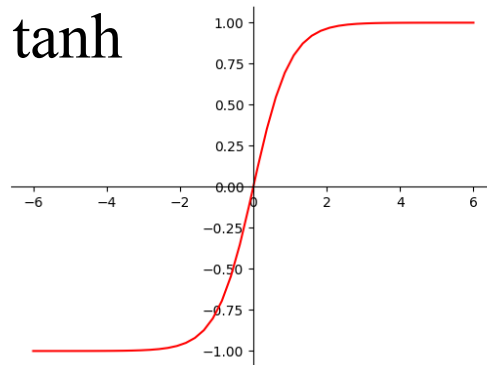
Leaky
ReLU



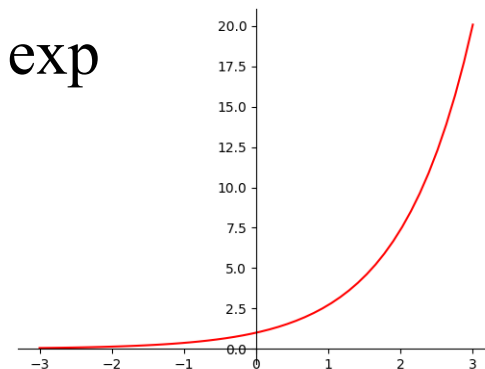
sigmoida



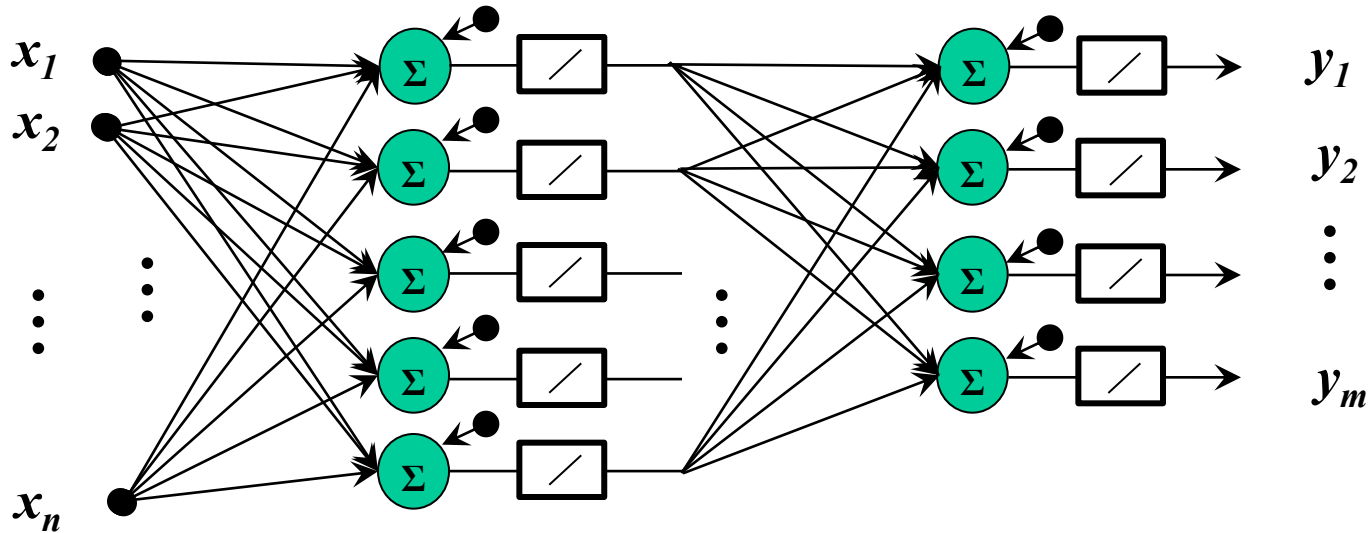
tanh



exp



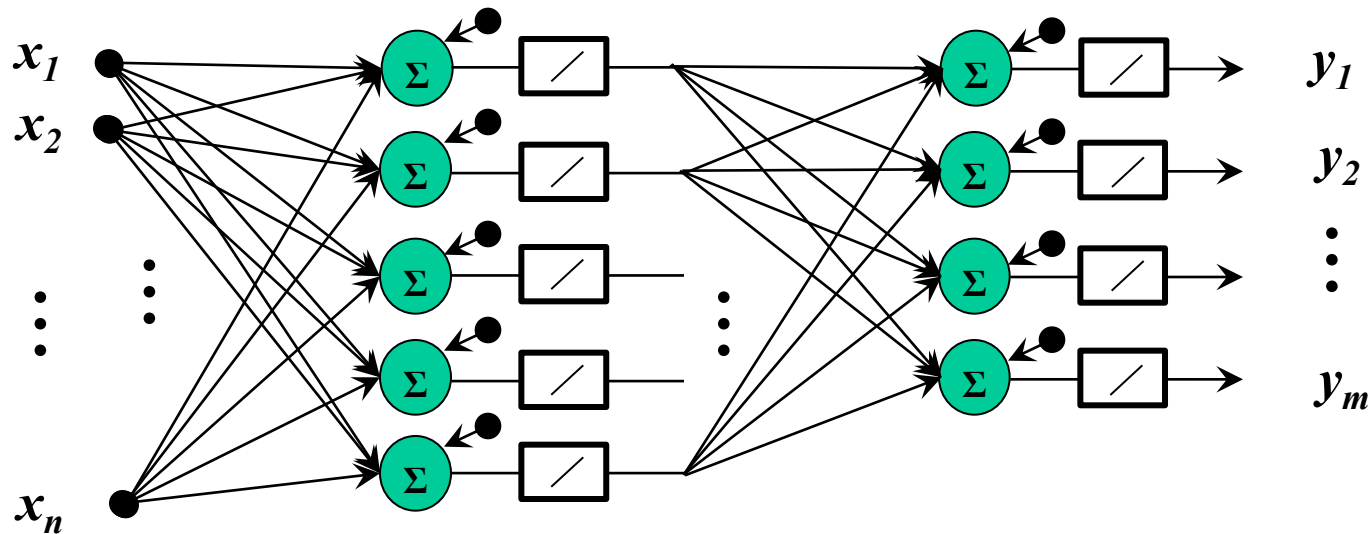
Perceptron



- jedna skrytá a jedna výstupná vrstva
- iba lineárna aktivácia
- možnosť trénovať len váhy výstupnej vrstvy

[1958 Rosenblatt]

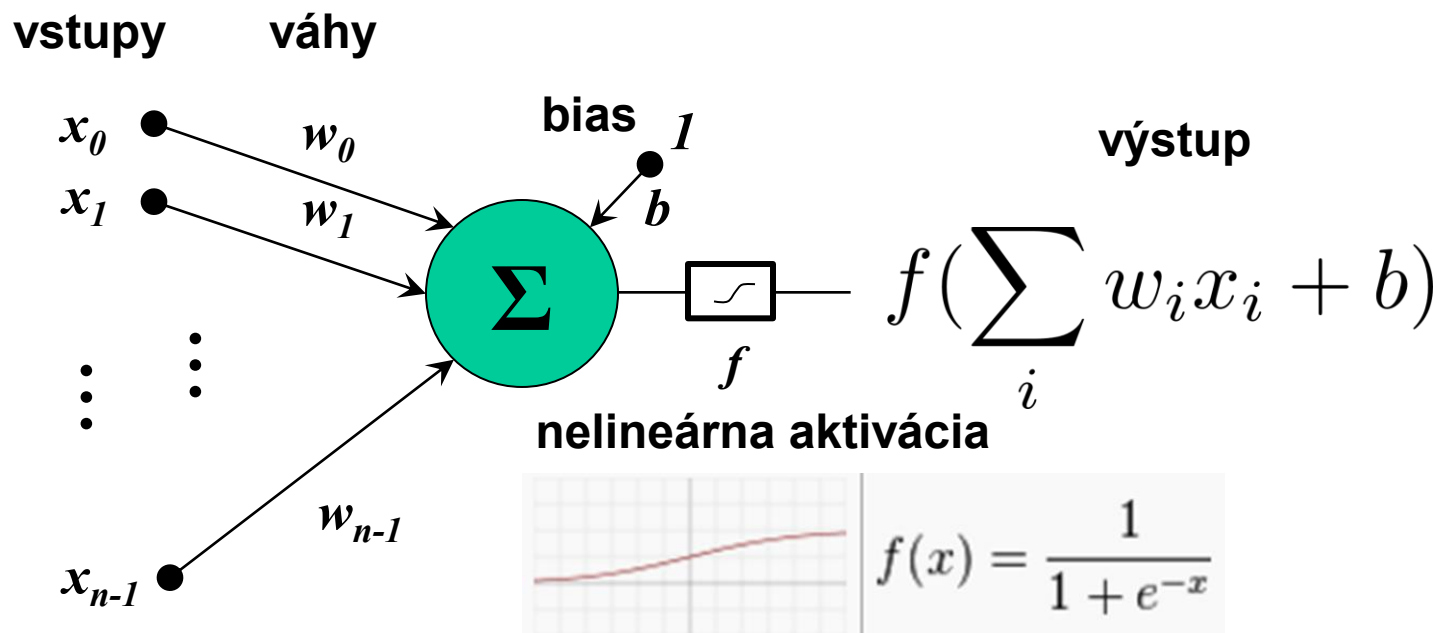
Perceptron



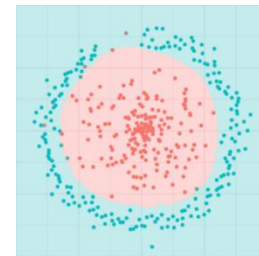
- analýza schopností perceptronu dokázala, že toho vie dosť málo
- neskôr sa však ukázalo, že jeho nevel'ká úprava situáciu dramaticky zmení

[1968 Minsky]

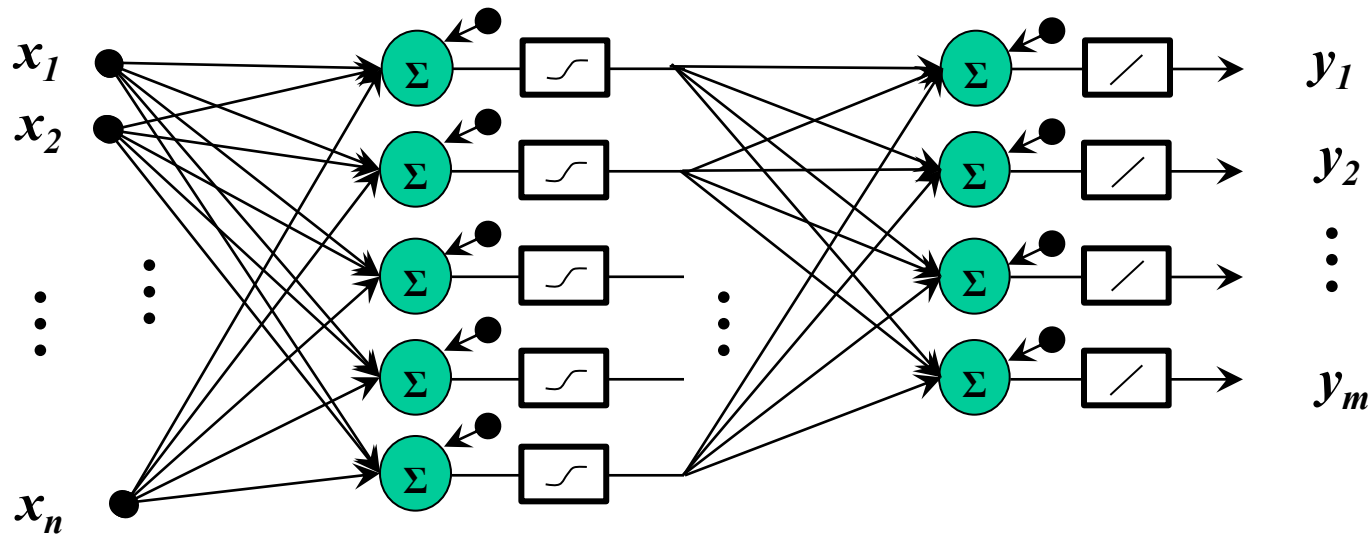
Neurón – Logistický regresor



- keď miesto lineárnej aktivácie použijeme logistickú funkciu (sigmoidu) alebo hyperbolický tangens, trénovanie jedného takéhoto neurónu zodpovedá logistickej regresii a potenciál neurónu byť stavebnou jednotkou siete sa dramaticky zlepší



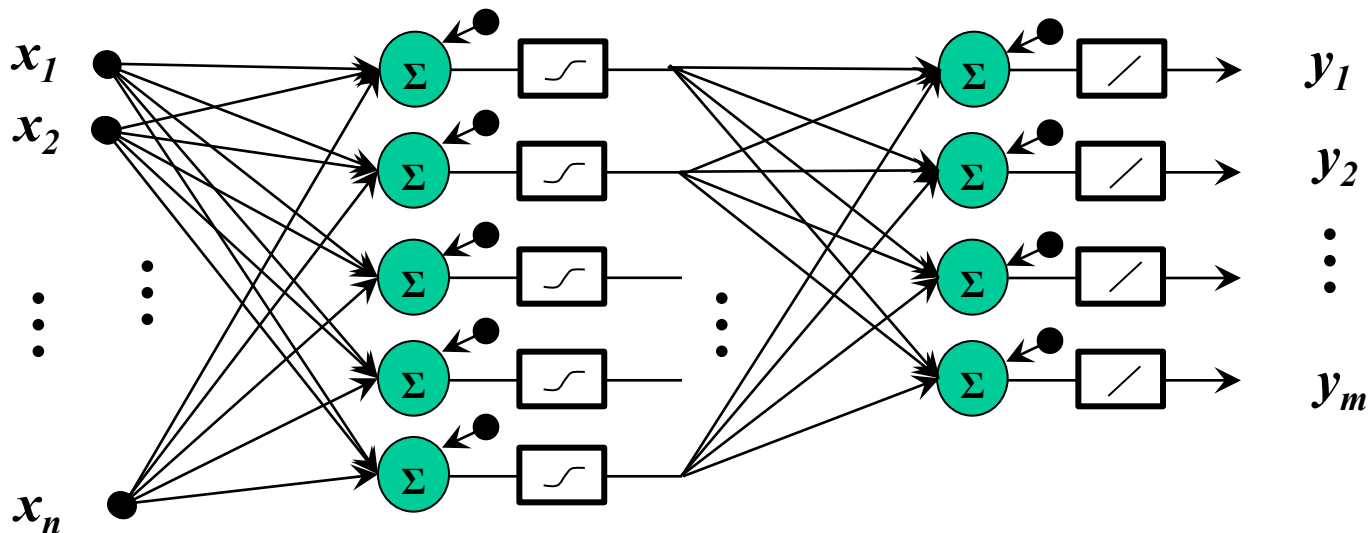
Perceptron + nelinearita + BP



- Algoritmus spätného šírenia (Back propagation) – pomocou pravidla derivácie zloženej funkcie [Leibnitz 1676] na trénovanie algoritmom spätnej propagácie (back propagation)

[1986 Rumelhart, Hinton & Williams]

Univerzálny aproximátor

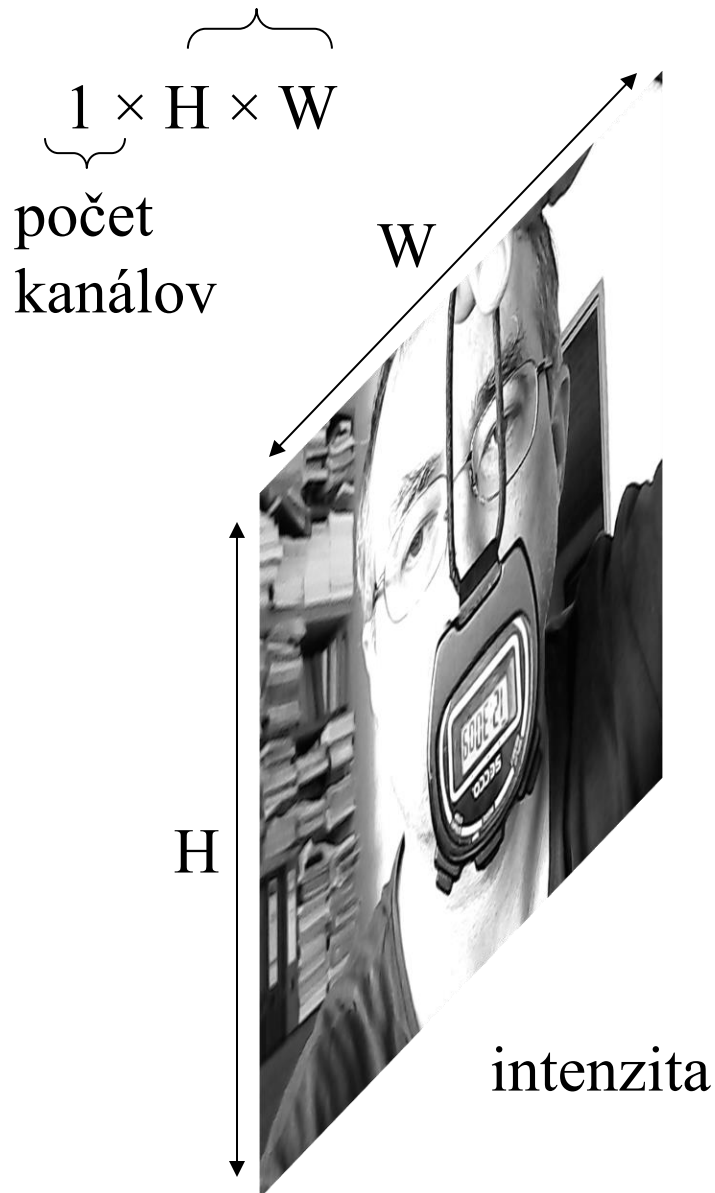


- matematicky bolo dokázané, že perceptron sa teoreticky dokáže s ľubovoľnou presnosťou naučiť akúkoľvek rovnomerne spojitú funkciu [1989 Cybenko] [1989 Hornik]
- prakticky sa to však podarilo uskutočniť len pre vstupy pomerne malej dimenzie, pre naše obrazové dáta s dimenziou $519168 = 416 \times 416 \times 3$ je to nepoužiteľné

Plán

- Potrebujeme nejako dimenziu obrazu zmenšiť bez toho, že by sme stratili jeho informačnú hodnotu
- Dnes si povieme: ako obraz spracúvať (pomocou kernelov)
- Nabudúce: ako zredukovať jeho dimenziu
- Na úvod: ako obraz reprezentujeme

Čiernobiely obraz



rozsah:

0 .. 255

0.0 .. 1.0

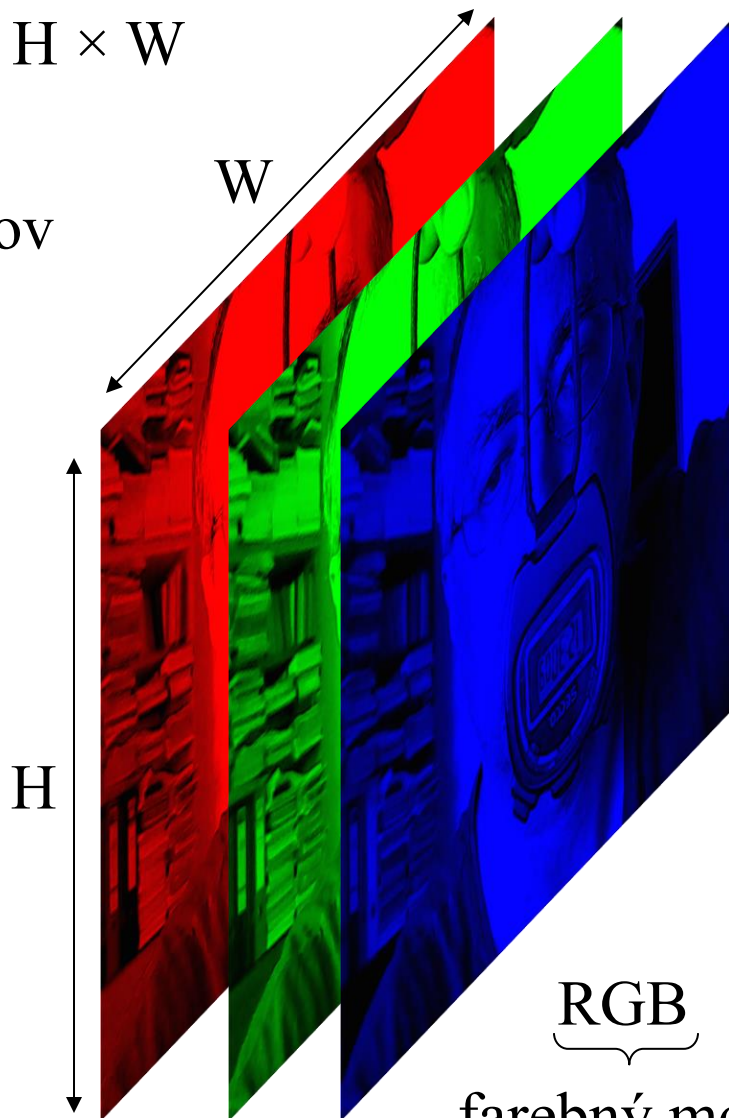
-1.0 .. 1.0

-0.5 .. 0.5

-3 .. 3

Farebný obraz

rozlíšenie
 $C \times H \times W$
počet
kanálov



$\underbrace{\text{RGB}}$
farebný model

rozsah:

0 .. 255

0.0 .. 1.0

-1.0 .. 1.0

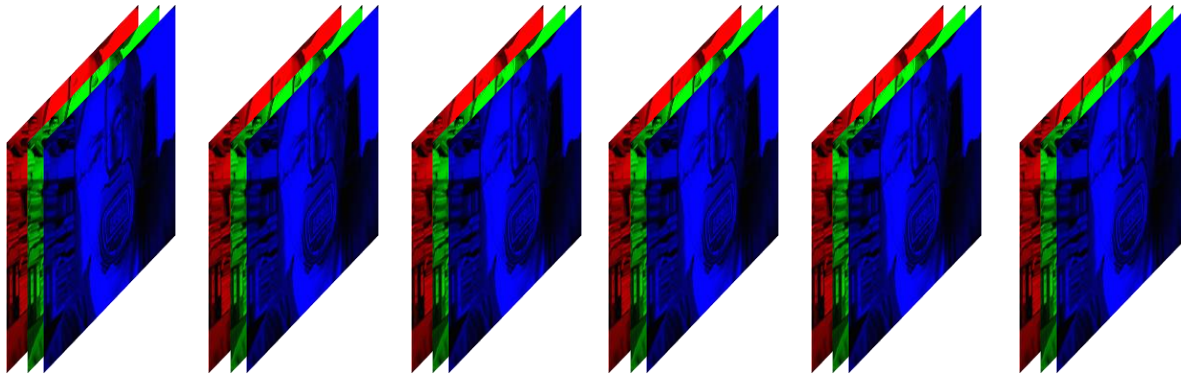
-0.5 .. 0.5

-3 .. 3



Dávka

$$B \times C \times H \times W$$



Kernel

240x320x1



1x1x1

1.5

weight

0.15

bias

240x320x1



intenzita x weight + bias

Kernel

240x320x1



1x1x1

1.5
weight

0

bias

240x320x1



váha je kontrast

Kernel

240x320x1



1x1x1

1.0
weight

0.15
bias

bias je jas

240x320x1



Convolutional Layer

320x240x1
input
values

input
dimension
76800



1x1x1
weight



neurons share
1 weight and
1 bias

the layer contains
76800 neurons
but has 2
parameters only

1 convolutional layer
320 x 240 neurons

320x240x1
output
values

Training



sample input



1.5 contrast
weight

0.15 brightness
bias



sample output

Kernel

240x320x3



3x3x3

	1/3	0	1/3	
	1/3	0	1/3	
-1/3	0	1/3	3	
-2/3	0	2/3	3	
-1/3	0	1/3	3	

weights

+0

bias

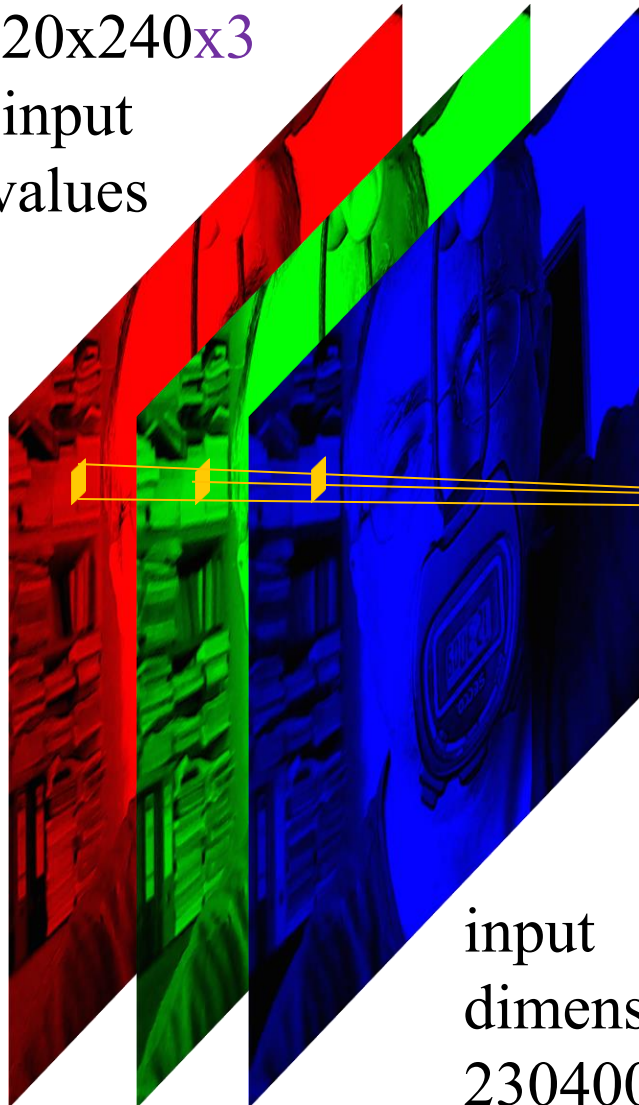
240x320x1



Sobel (vertical) kernel provides vertical edges

Convolutional Layer

320x240x3
input
values

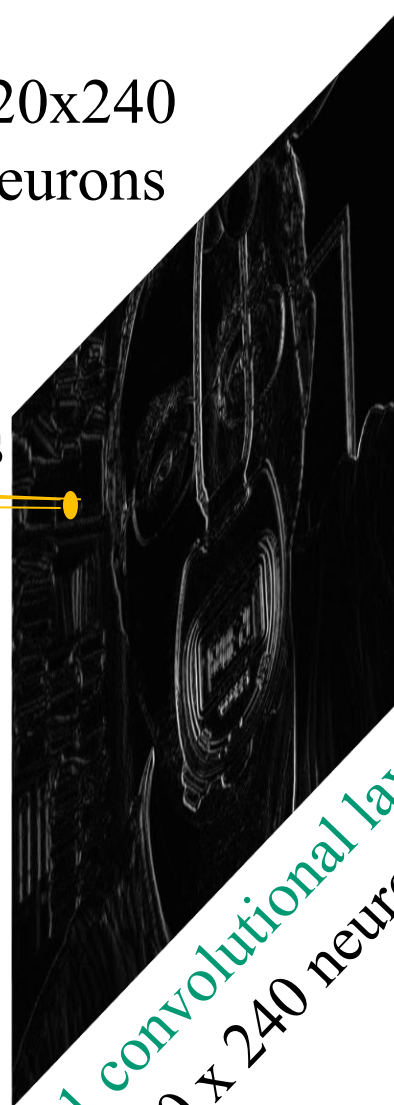


320x240
neurons

3x3x3
weights

1 bias

input
dimension
230400



neurons share
27 weights and
1 bias

the layer contains
76800 neurons
and 28 parameters

320x240x1
output
values

1 convolutional layer
320 x 240 neurons

activation: linear

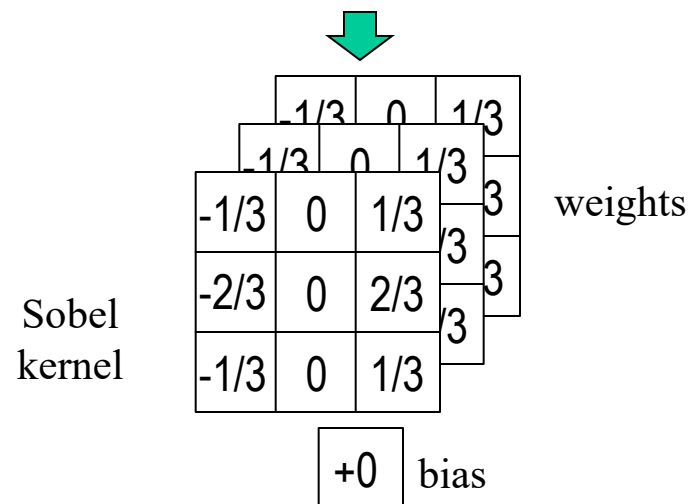
Training



sample input



sample output



Blok konvolučných vrstiev

320x240x3

vstupných
hodnôt

3x3x3

320x240x3

výstupných
hodnôt

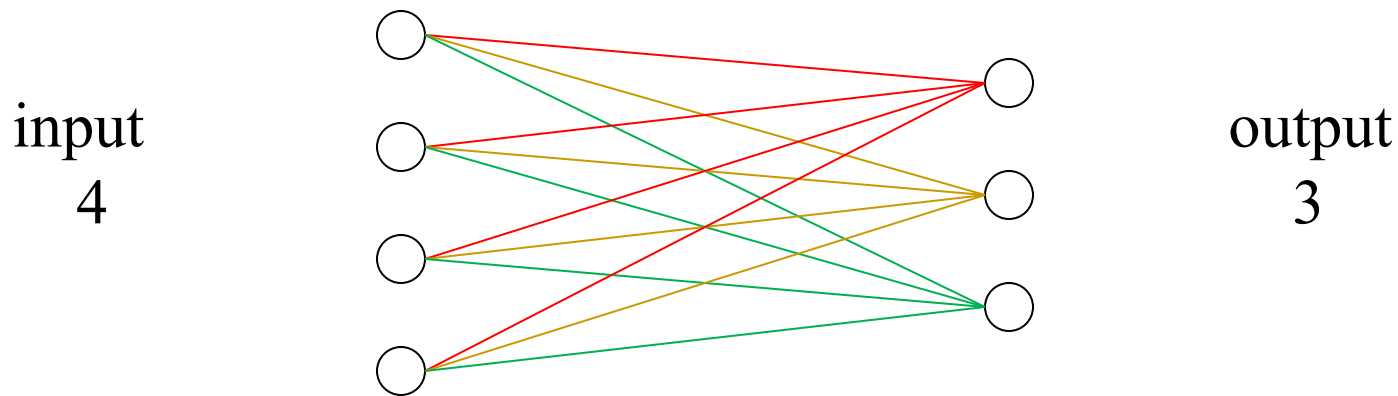
Inicializácia váh

- Defaultnou inicializaciou v Pytorch je Kaimingova distribúcia:

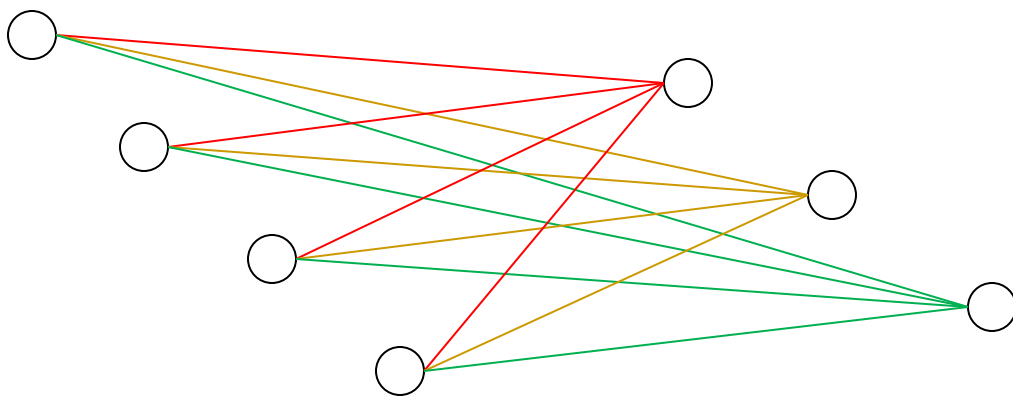
$$U[-x, x] \text{ kde } x = \sqrt{\frac{6}{N}}$$

kde N je počet vstupov a M počet neurónov

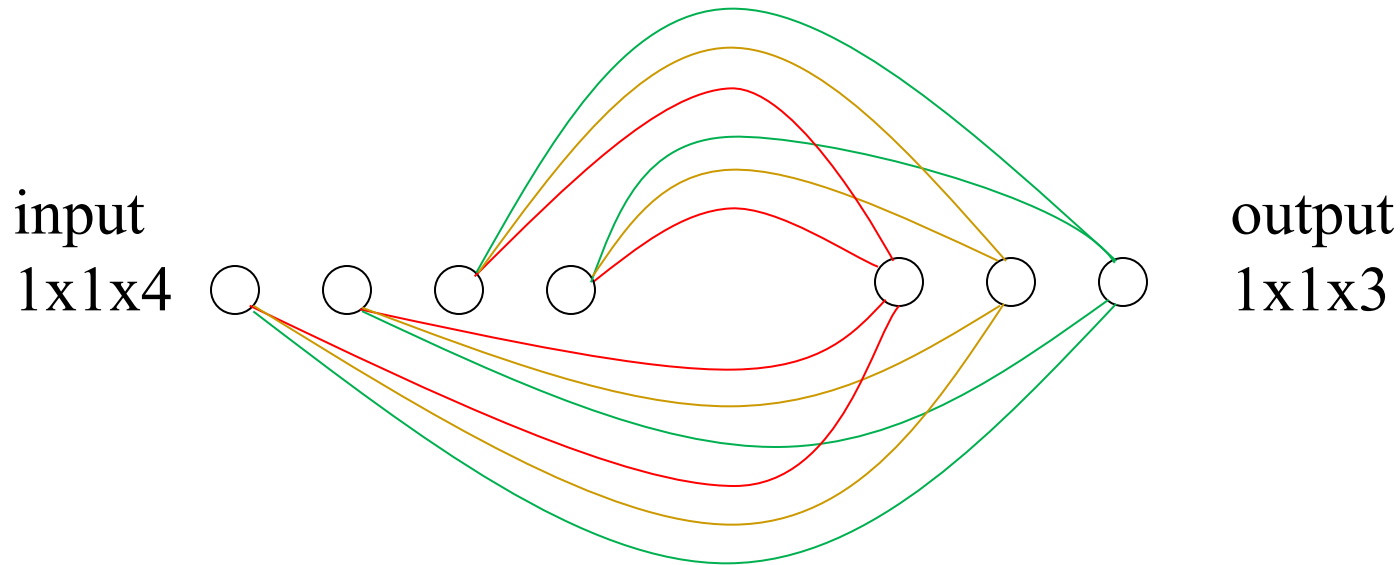
Fully – Connected layer (Linear) s N vstupmi a M neurónmi ($N \times M + M$ parametrov) zodpovedá bloku M konvolučných vrstiev prepojených na vstup $1 \times 1 \times N$ (rozlíšenie 1×1 a N kanálov) s kernelom 1×1



Fully – Connected layer (Linear) s N vstupmi a M neurónmi ($N \times M + M$ parametrov) zodpovedá bloku M konvolučných vrstiev prepojených na vstup $1 \times 1 \times N$ (rozlíšenie 1×1 a N kanálov) s kernelom 1×1

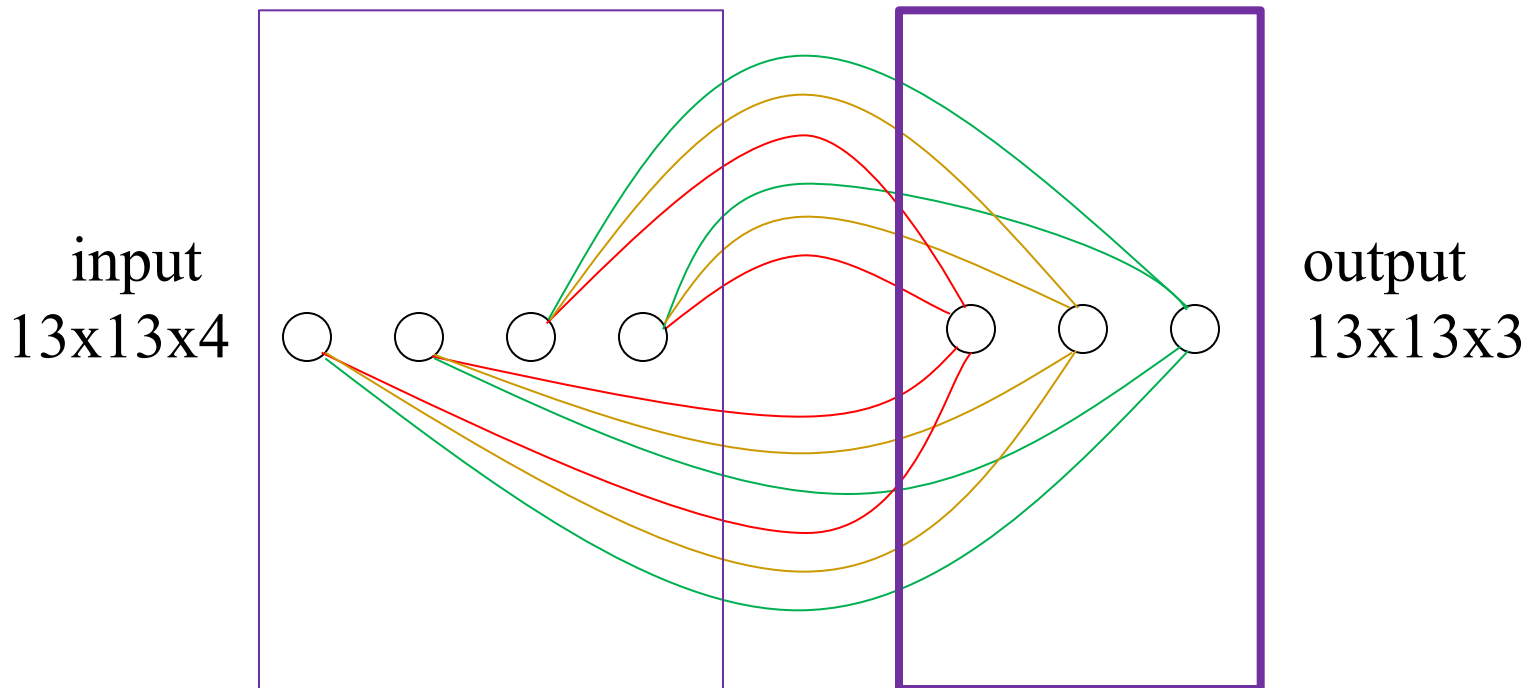


Fully – Connected layer (Linear) s N vstupmi a M neurónmi ($N \times M + M$ parametrov) zodpovedá bloku M konvolučných vrstiev prepojených na vstup $1 \times 1 \times N$ (rozlíšenie 1×1 a N kanálov) s kernelom 1×1



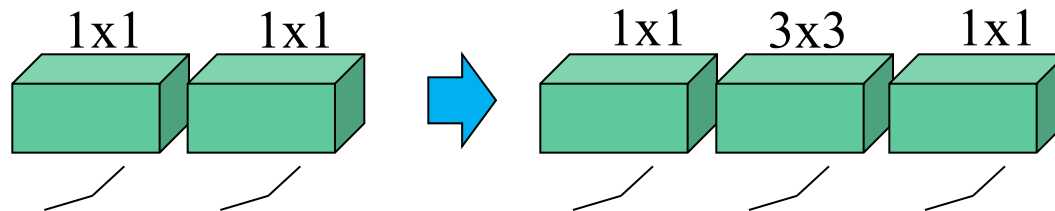
Ked' teraz ako vstup vložíme niečo s väčším resolution, napríklad 13x13x4, spustíme 169 paralelne bežiacich perceptrónov, ktoré zdieľajú váhy a biázy

block of 3 convolutional
layers with kernel 1x1



Čo je vlastne CNN?

- Čo ak teraz použijeme kernel väčší ako 1×1 , napríklad 3×3 ?
- Zabezpečí to výmenu informácií medzi susednými paralelne bežiacimi perceptrónmi.



Čo je vlastne CNN?

- CNN je teda sieť, kde paralelne púšťame rovnaké spolupracujúce perceptrony nad rôznymi miestami obrazu
- Tým premieňame obraz (mapa farieb) na mapu príznačkov

DNN

- Povalením perceptronov sa značne zväčšuje počet vrstiev v sieti
- Preto sa takýmto sieťam hovorí aj „hlboké“ - Deep NN

Fully-convolutional Network

- DNN kde sú všetky Linear nahradené Conv2D sú fully-convolutional
- Tieto siete nemajú (na rozdiel od iných DNN) pevné rozlíšenie vstupu
- Avšak vždy sú trénované na určité pevné rozlíšenie